Programa:	TurboNote 5.4
Protección:	Time-Trial + Nagscreen
Objetivos:	Que el programa corra en "Modo Registered"
URL descarga:	http://turbonote.com
Dificultad:	Fácil
Herramientas:	OllyDbg + Resource Hacker.
Cracker:	SyXe'05 [hAcK-c0d3d] [CracksLatinos] Fecha: 14-09-2005

Hola a todos una vez más..!!

Es el cuarto tutorial de hoy y me encuentro algo cansado, la verdad es que tenía ganas de hacer algo, me he sentado y de momento no lo he dejado, llevo una sesión bastante larguita, bueno es mi pasión así que no me sabe mal, supongo que a vosotros os pasará igual, hay veces que aún no he llegado a casa y voy loco por hacerlo y arrancar Olly y hacer algo ¡lo que sea! Imagino que a muchos de vosotros el cracking os hace sentir 'vivos' como me pasa a mi ;DDD

Bueno menos divagaciones espirituales y al meollo!!

Antes de hacer nada abrimos como viene siendo habitual el exe con PEiD a ver que tal:

Microsoft Visual C++ 7.0

Así que compilado en Visual C++ de compresión tampoco nada, cuatro de cuatro, los últimos que he mirado vienen así tal cual simempaquetar, esto además no es VisualBasic menos mal, jeje.. no es que me desagrade pero prefiero cosas de este estilo.

Lo cargamos en Olly pero ya, luego damos RUN dejándolo correr libre, miramos en el About y en los menús y no vemos la opción de Registro, en la barra de tareas hay un icono de la aplicación, picamos en él con el botón derecho y elegimos "General Settings…" en la última pestaña encontramos la opción "Register TurboNote+" picamos allí y tenemos la opción de poner un serial nadamás ni nombre ni nada, ponemos uno y aceptamos pero se cierra el cuadro sin más, ni mensaje ni nada ¿? Vale…

Volvemos a Olly y buscamos las referencias a cadenas, vemos esta zona:

```
00401B1A  |.  E8 61090100   CALL tbnote.00412480                    ; \tbnote.00412480
00401B1F  |.  83C4 08       ADD ESP,8
00401B22  |.  85C0          TEST EAX,EAX
00401B24  |.  0F84 B8000000 JE tbnote.00401BE2
. . .                       . . .
00401BFF  |.  6A 00         PUSH 0                     ; |wParam = 0
00401C01  |.  6A 0C         PUSH 0C                    ; |Message = WM_SETTEXT
00401C03  |.  50            PUSH EAX                   ; |hWnd
00401C04  |.  FFD7          CALL EDI                   ; \SendMessageA
00401C06  |.  68 5C254800   PUSH tbnote.0048255C       ;  ASCII "NOT REGISTERED"
```

Parece ser que el serial (o lo que sea) se valida en :401B1A, si es válido retornará EAX==1 pues en caso contrario saltará directo a la cadena NOT REGISTERED, ponemos un BP en dicho CALL, luego pulsamos en Abput y Olly para, entramos en él y vemos que es bastante grandecito, tiene muchos subcall y estos los suyos propios, aquí sacar algo en claro puede ser complicado, pero bueno igual lo miraremos que es gratis ;DDD

Nada más entrar (unos CALL más abajo) encontramos:

```
0041254B  |.  68 40304800   PUSH tbnote.00483040  ;"SOFTWARE\SPIS Ltd\TurboNote\Registration"
00412550  |.  68 02000080   PUSH 80000002
00412555  |.  8D8D 88FEFFFF LEA ECX,DWORD PTR SS:[EBP-178]
0041255B  |.  E8 90C40400   CALL tbnote.0045E9F0
```

Está claro que va a abrir la calve de registro que por otro lado no existe, de momento la creamos por si acaso, y seguimos traceando… pero no mucho mirad esto:

```
00412594  |.  E8 D7F40300   CALL tbnote.00451A70     ;mira si existe tbnote.tn
00412599  |.  83C4 04       ADD ESP,4
0041259C  |.  85C0          TEST EAX,EAX
0041259E  |.  0F84 EC010000 JE tbnote.00412790       ;si salta vamos a ERROR!!
```

Pongo las notas para que quede más aclarado lo que va haciendo ya que este CALL es enorme ;) si entramos en este SUBCALL encontramos:

```
00451A8E  |.  50            PUSH EAX                 ;/pFindFileData
00451A8F  |.  51            PUSH ECX                 ;|FileName = "tbnote.tn"
00451A90  |.  FF15 14224800 CALL DWORD PTR DS:[<&KERNEL32.FindFirstF>; \FindFirstFileA
```

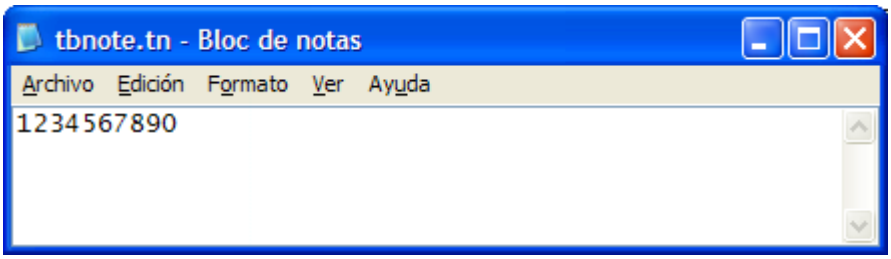
Está buscando sin duda alguna un archivo llamado "tbnote.tn" supongo que deberá estar en su mismo directorio, pues nada creamos uno a medida y se lo ponemos… ahora sigamos con el traceo cuando el archivo exista este call a FindFirstFileA retornará en EAX un valor diferente de 0, después nos encontramos esta llamada:

```
004125CF  |.  E8 8CE40100   CALL tbnote.00430A60          ;mira el tamaño
004125D4  |.  83F8 05       CMP EAX,5                     ;debe ser al menos de 5 chars
004125D7  |.  73 14         JNB SHORT tbnote.004125ED     ;si es asi seguimos
```

Entremos en éste:

```
00430A62  |.  6A 00         PUSH 0            ;/pFileSizeHigh = NULL
00430A64  |.  50            PUSH EAX          ;|hFile = 00000104 (window)
00430A65  |.  FF15 E8204800 CALL DWORD PTR DS:[<&KERNEL32.GetFileSiz>; \GetFileSize
```

La API GetFileSize retorna en EAX el número de bytes del archivo en hexa, como el nuestro está vacío, rellenémoslo de este modo:



Ahora al pasar sobre la llamada a GetFileSize tenemos un bonito EAX==0Ah que son los 10 caracteres que pusimos, bien como lo compara con 5 pasamos de momento la primera restricción ahora ya salta en :4125D7 y evitamos esta zona:

```
004125D9  |.  8D8D 78FDFFFF LEA ECX,DWORD PTR SS:[EBP-288]
004125DF  |.  C645 FC 03    MOV BYTE PTR SS:[EBP-4],3
004125E3  |.  E8 88E40100   CALL tbnote.00430A70
004125E8  |.  E9 BF020000   JMP tbnote.004128AC            ;si salta vamos a ERROR!!
```

Seguimos y al poco llegamos a esta parte:

```
004126A4  |.  E8 47330300   CALL tbnote.004459F0
004126A9  |.  83C4 0C       ADD ESP,0C
004126AC  |.  85C0          TEST EAX,EAX
004126AE  |.  8B85 58FDFFFF MOV EAX,DWORD PTR SS:[EBP-2A8]
004126B4  |.  75 2A         JNZ SHORT tbnote.004126E0
```

Está claro que si no salta en :4126B4 salimos del CALL con un ERROR como una casa como vemos en los comentarios, deberemos entrar en este SUBCALL y ver porqué no salimos con EAX=!0

```
00445A34  |.  57            PUSH EDI                                ; /String = "1234567890"
00445A35  |.  FFD5          CALL EBP                                ; \lstrlenA
00445A37  |.  8BF0          MOV ESI,EAX
00445A39  |.  83FE 14       CMP ESI,14
00445A3C  |.  0F8C 35020000 JL tbnote.00445C77                      ;  vamos a ERROR!!
```

El primer motivo es este (pues el CALL es grandecito también) que tenemos 10 caracteres y ahora nos sube la restricción a 14h que son 20 decimal, pongamos por ejemplo:

code121212121212syxe

cada uno que lo ponga como desee este es el que elegi yo, total es para probar la rutina a ver como vamos, volvemos a picar en About y seguimos, no he de ir diciendo cada vez que debemos poner un BP en cada zona donde tengamos problemas para parar allí directamente sin tracear todo otra vez. Venga pulsamos en About y a tracear, hay varias restricciones que vamos pasando bien y por no hacer el tuto muy laaaaaargo no enumeraré todas sólo la que nos vayan dando problemas, el siguiente es este:

```
00445B9F  |.  E8 CC05FCFF   CALL tbnote.00406170
00445BA4  |.  3BC6          CMP EAX,ESI
00445BA6  |.  0F85 81000000 JNZ tbnote.00445C2D      ;aqui salta y nos MATA..!!
```

Bueno pues este "error" entre comillas he de reconocer que es el que más vueltas me ha hecho dar, a ver como lo explico esto, resutlta que al salir del call compara EAX con ESI así tal cual, entonces en la ventana de registros vemos su valor que es:

```
EAX==00000000       . . .
ESI==000000B3       /* 179 decimal */
```

Entramos en el CALL 406170 y vemos que falla aquí:

```
00406180  |.  57            PUSH EDI                                 ; /String
00406181  |.  33DB          XOR EBX,EBX                              ; |
00406183  |.  33F6          XOR ESI,ESI                              ; |
00406185  |.  FFD5          CALL EBP                                 ; \lstrlenA
00406187  |.  85C0          TEST EAX,EAX
00406189  |.  7E 13         JLE SHORT tbnote.0040619E      ;Aqui salta y nos TIRA..!!
```

El salto se produce y salimos con un ERROR monumental EAX==0, como vemos la llamada tiene que ver con la longitud del código, será el del archivo tbnote.tn? por si acaso lo hacemos un poco mayor, ponemos "code1212121212121212syxe" que al que le hemos añadido "1212" ahora probamos y… jeje., no salta y tenemos que EAX==3, vamos avanzando, cae en este loop ahora:

```
00406190  |>  0FBE0437      /MOVSX EAX,BYTE PTR DS:[EDI+ESI]
00406194  |. |03D8          |ADD EBX,EAX
00406196  |. |57            |PUSH EDI
00406197  |. |46            |INC ESI
00406198  |. |FFD5          |CALL EBP
0040619A  |. |3BF0          |CMP ESI,EAX
0040619C  |.^\7C F2         \JL SHORT tbnote.00406190
0040619E  |> \5F            POP EDI
0040619F  |.  5E            POP ESI
004061A0  |.  8BC3          MOV EAX,EBX              ;EAX pilla el valor de retorno..!!
004061A2  |.  5D            POP EBP
004061A3  |.  C705 1C7D4900>MOV DWORD PTR DS:[497D1C],35739C
004061AD  |.  25 FF000000   AND EAX,0FF
004061B2  |.  5B            POP EBX
004061B3  \.  C3            RETN
```

Subrayo las dos instrucciones clave de todo esto, primero se mete en un bucle donde va tomando caracteres ya veremos de donde, va acumulando el resultado parcial de cada pasada en EBX, vemos que los está sumando "ADD" luego este valor es el que pasa a EAX y es precisamente el valor de retorno del CALL, sabemos que deberá quedar EAX==0B3h, de momento queda EAX==64h y no nos alcanza a lo que buscamos, pues es en este punto donde más vueltas he dado ;DDD para empezar deberemos encontrar el lugar de donde está leyendo los caracteres en este loop, pues lo hace de [12ED30], pero ¿quién escribe estos datos aquí? He ahí la cuestión, lo que hice fue parar la ejecución un poco antes y ver si estaban los datos ya escritos, sino estaban poner un BPM on Write hasta encontrar el lugar dentro de estos CALL'S:

```
00445B5F  |.  E8 7C08FCFF   CALL tbnote.004063E0
      0040646B  |.  E8 70FCFFFF   CALL tbnote.004060E0
```

dentro de este SUBCALL encontramos la zona finalmente:

```
00406110  |> /8A1438        /MOV DL,BYTE PTR DS:[EAX+EDI]
00406113  |. |889406 430100>|MOV BYTE PTR DS:[ESI+EAX+143],DL
0040611A  |. |40            |INC EAX
0040611B  |. |3BC1          |CMP EAX,ECX
0040611D  |.^\7E F1         \JLE SHORT tbnote.00406110
```

pues en este bucle se escreiben los bytes estudiados, pero para complicar un poco más el tema resulta que los lee de otra variable, jooooder [12EB15] es el culpable, pues nada hacemos lo mismo hasta encontrar el punto clave:

```
004063A0  |>  8BB1 44010000 /MOV ESI,DWORD PTR DS:[ECX+144]
004063A6  |. |8A0417        |MOV AL,BYTE PTR DS:[EDI+EDX]        ;lee los bytes
004063A9  |. |32840E 480100>|XOR AL,BYTE PTR DS:[ESI+ECX+148]    ;los encripta
004063B0  |. |46            |INC ESI
004063B1  |. |81E6 03000080 |AND ESI,80000003
004063B7  |. |79 05         |JNS SHORT tbnote.004063BE
004063B9  |. |4E            |DEC ESI
004063BA  |. |83CE FC       |OR ESI,FFFFFFFC
004063BD  |. |46            |INC ESI
004063BE  |> |89B1 44010000 |MOV DWORD PTR DS:[ECX+144],ESI
004063C4  |. |8802          |MOV BYTE PTR DS:[EDX],AL
004063C6  |. |42            |INC EDX
004063C7  |. |4B            |DEC EBX
004063C8  |.^\75 D6         \JNZ SHORT tbnote.004063A0
```

Pues este loop tiene la culpa, aquí se guisa todo el tema este, la pregunta ahora es ¿de dónde saca estos valores? O sea, de donde los lee? Y qúe lee claro está, resulta que ahora lee los bytes de otra variable [3F434B] donde está la cadena encriptada "r‡^×mv×mv×mv×mv" que creo porque no lohe mirado que es parte de nuestro código encriptado (el de tbnote.tn). Tras mucho investigar y no es broma, encontré que para el código que hemos puesto:

code1212121212121212syxe

los bytes que parece estar leyendo son los que remarco ahora:

code1212121212121212syxe

Pues parece que estos para este código concreto estos son los bytes importantes, os pongo una mini_lista de la serie de pruebas que estuve haciendo antes de dar en el clavo:

Necesitamos: EAX==0B3h

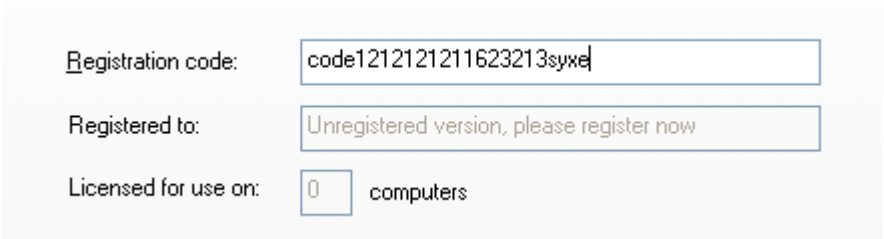
```
212 => EAX==64h
200 => EAX==26
201 => EAX==25
202 => EAX==24
210 => EAX==66
220 => EAX==83      no llegamos…!!
230 => EAX==E6      nos pasamos…!!
235 => EAX==E1
23ª => EAX==1A
236 => EAX==E0
237 => EAX==DF
238 => EAX==DE
239 => EAX==DD
240 => EAX==25
245 => EAX==20
250 => EAX==65
255 => EAX==60
260 => EAX==A5
270 => EAX==E5
280 => EAX==24
290 => EAX==64
260 => EAX==A5
360 => EAX==95
160 => EAX==B5
162 => EAX==B3…!!! Para código => code1212121211623213syxe
```

Increíblemente con este código nos dice Registered To: (aunque no sale nada al lado ¿?)
License for use on 37608 Computers

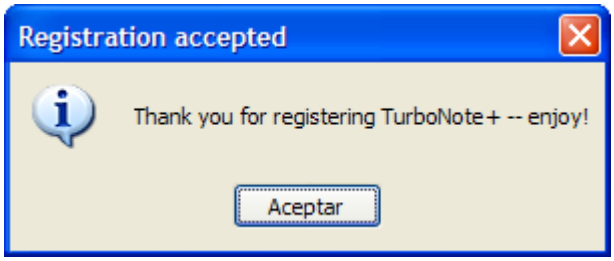
De verdad que no pongo la comedura de tarro que me pegué por no aburrir, al final los resultados fueron los que vemos en la lista que es una muestra de las pruebas que yo hice, desde luego que hay maneras mucho más elegantes para encontrar el serial, pero ya con esta costó, jeje.. no quiero imaginar tener que tracear todo el código investigando cada variable. En definitiva ponemos este serial en el archivo tbnote.tn y nos reconoce como usuarios registrados, parece ser que le pusimos lo que quería que le pusiéramos ;-)

Ahora un detalle estético, el "Buy Now…" ha desaparecido, el "NOT REGISTERED" también, esto no es problema pero no pone nuestro nombre ;-(después del trabajo y ni nuestro nombre nos pone, no hay derecho uno pasa un buen rato crackeando el programa y mira como nos lo pagan ;) por suerte para nosotros existen muy buenas herramientas que nos ayudan bastante, en este caso me refiero al Resource Hacker que nos va a servir para rematar la faena, es muy muy simple, abrimos el .exe y hacemos una búsqueda de la cadena "Registered" aparece tres veces, la tercera es la buena (nunca mejor dicho) que encontramos en la "String Table" sección 15/1033 allí cambiamos el "Registered to:" por "Licensed to: + nuestro_nombre" y esto mismo aparecerá en el About, jeje..

Decir que esto último no hacía falta porque el programa ya estaba registrado. Otra observación es que la rutina con la que nos hemospeleado es la misma que valida el serial (lógico) me refiero que si ponemos este serial:



que es el que hemos encontrado por decirlo de alguna manera y Pulsamos aceptar como el que pulsa no sé sin esperar nada ocurre ¡¡!!



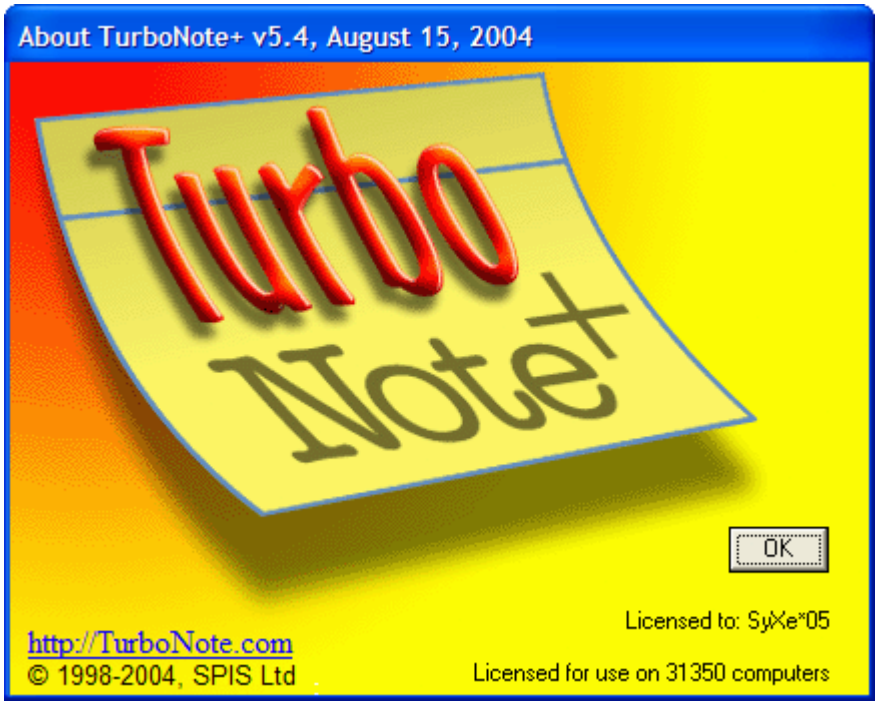
jodeeeeeeeeer si vale hasta como serial!! Esto si que no lo esperabamos, ahora creó un valor en el registro de Window$:

HKEY_LOCAL_MACHINE\SOFTWARE\SPIS Ltd\TurboNote\Registration
"Code"=",Ô□D□‹□<□q,ö/Ä,ð□Š□&□{,Ô/¤,ø,æ□†/´□□□&□b/Æ□□/ª,ï/²,Ô,ã□10□□A□9,ø□,□□,Ñ□R□i/›□b,ê"

Notad que está todo súper encriptado!

Otra cosa rara: Para que en el About salga nuestro nombre modificamos la cadena "Registered to:" por otra del tipo "Licensed to: SyXe'05" si no cambiamos el Registered por Licensed no aparece nuestro nombre al lado ¿?

Y terminamos con esto el trabajo comenzado, veamo como queda el About



jeje, a nuestro nombre y todo, hombre después de todo a uno también le gustan estas cosas ;DD

Ahora un apunte: Podemos borrar la clave "code" del registro y el programa correrá Registered, podemos eliminar el archivo tbnote.tn y también lo hará, pero nunca ambos indicadores a la vez porque pasa a modo Trial, peeeeeero para mi sorpresa borro ambos indicadores, le inserto el serial válido que encontramos, vemos que crea la clave del registro pero no el archivo tbnote.tb! como es posible? En cambio si le borramos todos los indicadores y le ponemos este archivo en su mismo directorio con el serial válido en su interior corre en modo Registered, alguna explicación ¿?

Bueno después de todo esto y del último interrogante podemos dar por terminado el tuto, todos los objetivos se cumplieron, al principio parecía que iba a ser más complicado de lo que realmente fue ;DDD

Si alguien me quiere localizar puede hacerlo aquí:

syxe00@yahoo.es
syxe00@hotmail.com